What is a Simulation Study?

For this activity, we want to discuss what a simulation study is, how to design one, and explore commenting code and other best practices for writing R code (the lessons apply to most other coding languages).

Best Practices for Coding in R

Basically, I've just assembled some resources for you to examine at your leisure. There are many other similar pages out there.

https://www.r-bloggers.com/r-code-best-practices/

This first link has lots of links to other resources at the bottom of it's page.

Tidyverse Style Guide

Google's Style Guide

In regards to commenting, you don't need to comment EVERY line, but the idea is to comment chunks, functions, etc. with their use/purpose. This helps both you and your readers. You also want to keep chunks relatively small - they become hard to parse when they get large (like paragraphs). Don't try to do too much in each chunk.

I strongly recommend the following when it comes to your course projects (and other future work) in terms of coding, especially if you are running a simulation study.

- Render to pdf early and often. Don't wait till the last minute to render/compile.
- Write code in small chunks to check that it works. A large chunk that doesn't run is harder to debug.
- Intentionally break code INTO small chunks. You can always put it into a larger chunk later if needed.
- Caching is useful, but be careful about pushing/committing said pieces to your repos. The limit is 100MB.
- Learn RMarkdown formatting and use it. There's a cheatsheet from RStudio.

- Don't submit at the last minute. If it's late, it's late. Git gives us a time record! Submit EARLY.
- Make sure you set seeds. You need a reproducible submission for the project.

Simulation Uses - Toy Examples versus Simulation Studies

Simulation can be used in many ways that are useful for your audience. For the final project, you are choosing between an application and a simulation study, so we want to explore what a simulation study is (and isn't).

You may use simulation to generate a toy data set or example to illustrate a point in your exposition. For example, this is what I did in the Ch. 15 activity you explored. I generated 100 p-values from a particular setup so we could explore them, and know the truth while we did so. This is called a toy example. Similarly, trying out a new method on a really simple data set, such as linear regression on the iris or penguins data sets is usually considered a toy example too. It would be similar for trying the examples at the end of R help files. Sometimes those are on real data, but could be simulated too. But they don't evaluate anything - they are designed to show something in a context of a single example, usually without any replication.

I encourage you to include a toy example in your exposition section. It can help to have this as it's usually an easier data set to work with and you can use it to demonstrate how functions work from R packages, etc. This is encouraged whether you are doing an application or simulation study for the second component. Think about how you've learned methods in class. There's usually an example, right? This is that example. If you go the application route, you can have the toy example, but then the idea is the application is a much more complicated example, where you don't know what will happen / will applying the method work well, etc. For toy examples, we usually know the methods "work'' which is why we use them as demonstrations.

But how do toy examples and simulation studies differ then?

A simulation study kicks the simulation up considerably. Here, the idea is you have something you want to show or do (such as compare two methods), and you know that just doing it on a single simulated data set is nice but that's just a single "data point''. You need to replicate. Instead of doing a single simulation run, you need more of them. And there's usually something you are varying (like in an experiment) in order to assess it's impact on the outcome.

Here are two examples of simulation studies from my own work:

A colleague and I have a nonparametric test for interaction in the two-way layout (ANOVA). To show how it performs against competitors, we

- identified multiple competitors we wanted to check,
- chose two main forms of interaction we wanted to study,

- chose different effect sizes for the two factors involved,
- picked different distributions for the errors in the models, and
- picked different sized layouts to use.

At EACH combination of those, we generated multiple data sets and computed the power for each competing test procedure. Even if there were only 2 levels of each of these, that's 2^5 = 32 different parameter sets. For each one, you can imagine needing to generate 100 or more datasets (we used 100,000 for developing the distribution of our statistic).

My graduate thesis advisor and I developed a new covariance estimator to use in high-dimensional settings. To show how it performed against competitors, we

- identified multiple competitors we wanted to check,
- chose two main covariance structures we wanted to study,
- chose different strengths of relationships in the covariance matrices to use, and
- picked three values for p, the number of variables involved.

At EACH combination of those, we generated covariance matrices, applied the estimators, and got norms comparing the estimates to the truth (which we knew because that's how we generated the matrices). Again, there are many replicates at each particular combination of parameter values.

If you are aiming to do a simulation study for your project, I encourage you to think about what you want to show, and only choose a few things to have different levels of to compare. For example, perhaps you apply 2 different procedures on data sets generated with 3 different percentages of missingness, using some underlying common model with a fixed number of variables involved. That's 6 combinations to compare.

Developing a Simulation Study

There's a lot to think about when trying to develop a simulation study.

What things/quantities might you want to study? It will depend on the setting / methods you are examining. You could check things like Type I error, power, or confidence interval coverage, for example. The simulation study below is in the setting of nonparametric statistics. It compares three test procedures in the one-sample setting examining measures of center - the parametric t-test, which you should know, and 2 nonparametric tests (the sign test and signed rank test).

From theory, we know that there are situations where the t-test does not perform well compared to the Sign Test or Signed Rank test, and vice versa. To investigate and demonstrate this to ourselves, we'll walk through setting up a simulation study, and go through the steps associated with that below.

What do we need to do in order to set up this simulation study?

- We need to know how to run the 3 tests on a single data set.
- We need to figure out how to generate data sets under different conditions to study behavior.
- We need to know what settings / values we're using to generate our truth and any relevant info for each.
- We need to figure out what to save from each application of each test on each data set in order to demonstrate our point from the study.

If you are doing a simulation study for the project, feel free to ask for assistance with this part - the setup can be the most challenging component. But you could follow a similar process to what is here (and is why we are going over this). In a general framework, that looks something like this:

- What methods am I trying to compare?
- Do I know how to implement each method in R (or whatever language) already?
- Do I know what values I will be varying / changing to study the impact of?
- Do I know how to generate my truth?
- Is there anything I'm not varying that I need to specify or have more information for?
- What do we save from each replication to demonstrate our point from the study?

Running the three tests

First, let's just see how to run all three test procedures on a set of observations. You can imagine these are the differences or just a set of values (i.e. paired setting or just one-sample setting). All these are run as two-sided examples. That could be changed with the *alternative* option, but we'll run everything here two-sided.

```
fakedata <- c(1:50) #1:50 as our data
t.test(~ fakedata, mu = 25) #parametric t-test
```

```
    One Sample t-test

data:  fakedata
t = 0.24254, df = 49, p-value = 0.8094
alternative hypothesis: true mean is not equal to 25
95 percent confidence interval:
 21.35715 29.64285
sample estimates:
```

```
mean of x
     25.5
```

```r
SIGN.test(fakedata, md = 25) #sign test
```

```
    One-sample Sign-Test

data:  fakedata
s = 25, p-value = 1
alternative hypothesis: true median is not equal to 25
95 percent confidence interval:
 18.53511 32.46489
sample estimates:
median of x
       25.5

Achieved and Interpolated Confidence Intervals:

                  Conf.Level  L.E.pt  U.E.pt
Lower Achieved CI     0.9351 19.0000 32.0000
Interpolated CI       0.9500 18.5351 32.4649
Upper Achieved CI     0.9672 18.0000 33.0000
```

```r
wilcox.test(fakedata, mu = 25) #signed rank test
```

```
Warning in wilcox.test.default(fakedata, mu = 25): cannot compute exact p-value
with ties

Warning in wilcox.test.default(fakedata, mu = 25): cannot compute exact p-value
with zeroes


    Wilcoxon signed rank test with continuity correction

data:  fakedata
V = 637, p-value = 0.8113
alternative hypothesis: true location is not equal to 25
```

Now, to do our simulation study and compare the methods, we're going to need to *SAVE* some values from the output of the tests. What might be useful values to save? (Last item above in list.)

Discuss.

It would probably be useful to save the test statistics and p-values (primarily p-values). Let's consider how that can be done. We can rerun the tests and SAVE the output in order to see what pieces are actually computed by R when the test is run. These pieces can be extracted separately.

```
tresult <- t.test(~ fakedata, mu = 25) #parametric t-test
names(tresult)
```

```
 [1] "statistic"   "parameter"   "p.value"     "conf.int"    "estimate"
 [6] "null.value"  "stderr"      "alternative" "method"      "data.name"
```

```
signresult <- SIGN.test(fakedata, md = 25) #sign test
names(signresult)
```

```
 [1] "statistic"            "parameter"            "p.value"
 [4] "conf.int"             "estimate"             "null.value"
 [7] "alternative"          "method"               "data.name"
[10] "Confidence.Intervals"
```

```
srresult <- wilcox.test(fakedata, mu = 25) #signed rank test
```

```
Warning in wilcox.test.default(fakedata, mu = 25): cannot compute exact p-value
with ties
```

```
Warning in wilcox.test.default(fakedata, mu = 25): cannot compute exact p-value
with zeroes
```

```
names(srresult)
```

```
[1] "statistic"   "parameter"   "p.value"     "null.value"  "alternative"
[6] "method"      "data.name"
```

So, this means that I could extract all three p-values from the saved tests like this:

```
tresult$p.value
```

```
[1] 0.8093775
```

```
signresult$p.value
```

```
[1] 1
```

```
srresult$p.value
```

```
[1] 0.8112835
```

(You should be able to do this for similar pieces of similar objects.)

Even better, I don't have to SAVE the test output to get the p-values. I can get them like this:

```
t.test(~ fakedata, mu = 25)$p.value
```

```
[1] 0.8093775
```

```
SIGN.test(fakedata, md = 25)$p.value
```

```
[1] 1
```

```
wilcox.test(fakedata, mu = 25)$p.value
```

```
Warning in wilcox.test.default(fakedata, mu = 25): cannot compute exact p-value
with ties
```

```
Warning in wilcox.test.default(fakedata, mu = 25): cannot compute exact p-value
with zeroes
```

```
[1] 0.8112835
```

Ok. So now we have a way to extract p-values quickly from a test. We want to combine this with simulating MANY data sets, using different distributions for the data to see how the different tests behave. (There's a bit more to this, including understanding effect size, but we should be able to have some fun with just this understanding.)

Simulating Data from a Normal Distribution

Here, we look at an example simulating data from the normal distribution - this is a condition we could say we want to explore. The code below samples 10 observations from a normal distribution with mean 20 and sd 2, and saves them as x.

```
x <- rnorm(10, 20, 2); x
```

```
 [1] 19.78012 20.83021 22.16607 22.64611 22.01031 24.01910 17.96109 19.39871
 [9] 23.91262 19.90954
```

We could feed x into our different tests, along with a hypothesized mean, get the p-values, and compare the results. For the study, we need to simulate MANY different but similar x's (i.e. same "condition'') and repeat this process to better understand the behavior of the tests.

Here's an example:

```
# Set Useful Values
reps <- 1000 #number of repetitions
truemu <- 40
truesd <- 3
# if truemu and testcenter are not =, we hope our tests detect this
testcenter <- 37 #center to test for
n <- 25 #sample size
set.seed(1001) #for reproducibility

#Initialize storage vectors
tpvals <- rep(0, reps)
spvals <- rep(0, reps)
srpvals <- rep(0, reps)

#Generate random data, do tests, save p-values
for(i in 1:reps){
  x<-rnorm(n, truemu, truesd)
  tpvals[i] <- t.test(~x, mu = testcenter)$p.value
  spvals[i] <- SIGN.test(x, md = testcenter)$p.value
  srpvals[i] <- wilcox.test(x, mu = testcenter)$p.value
}
```

What does running this give us? Well, we get a set of 1000 p-values each from the three different tests (1000 data sets all run through the three tests). We can figure out how often

each test rejected the null, and see which test is performing better. Here, we know the true mean is 40, and we were testing a two-sided alternative with a mean of 37. So, we'd like to think the tests would reject the null.

```
sum(tpvals <= 0.05)/reps
```

```
[1] 0.999
```

```
sum(spvals <= 0.05)/reps
```

```
[1] 0.956
```

```
sum(srpvals <= 0.05)/reps
```

```
[1] 0.999
```

We see that 99.9% of t-tests and Signed rank tests rejected the null, but only 95.6% of sign tests did. The output is the fraction of p-values less than or equal to 0.05.

How can we make this easier to implement?

Here is the same code (pieces combined), and turned into a function (yes, you will probably want functions if doing your own simulation study, but you may need different functions for your different conditions, depending on how complicated it is). You run the main chunk once to create the function, and then you can run it quickly with different inputs in chunks after it.

```
simnormal <- function(locationinput, scaleinput, testcenterinput, ninput,
                      repsinput = 1000){
# Set Useful Values
reps <- repsinput #number of repetitions
location <- locationinput
scale <- scaleinput
testcenter <- testcenterinput #center to test for
n <- ninput #sample size

#Initialize storage vectors
tpvals <- rep(0,reps)
spvals <- rep(0,reps)
srpvals <- rep(0,reps)
```

```
#Generate random data, do tests, save p-values
for(i in 1:reps){
  x <- rnorm(n, location, scale)
  tpvals[i] <- t.test(~x, mu = testcenter)$p.value
  spvals[i] <- SIGN.test(x, md = testcenter)$p.value
  srpvals[i] <- wilcox.test(x, mu = testcenter)$p.value
}

output <- c(sum(tpvals <= 0.05)/reps,
            sum(spvals <= 0.05)/reps,
            sum(srpvals <= 0.05)/reps)
names(output) <- c("Ttest", "Sign", "SignedRank")
output
}
```

The output is in the order of t-test, sign test, and lastly, signed rank test, and it produces the fractions of p-values less than or equal to 0.05. Some short labels were added to help with keeping the order straight. You could envision fancier functions that took in optional alphas to compare to instead of just 0.05.

```
set.seed(1001)
simnormal(40, 3, 37, 25) #mean, sd, testcenter, n are the inputs
```

```
     Ttest       Sign SignedRank
     0.999      0.956      0.999
```

```
simnormal(20, 3, 27, 25)
```

```
     Ttest       Sign SignedRank
         1          1          1
```

```
simnormal(40, 7, 37, 25)
```

```
     Ttest       Sign SignedRank
     0.551      0.358      0.518
```

```
simnormal(40, 3, 37, 50)
```

```
     Ttest       Sign SignedRank
     1.000      0.998      1.000
```

```
simnormal(40, 3, 37, 10)
```

```
     Ttest       Sign SignedRank
     0.791      0.522      0.783
```

```
simnormal(40, 1, 37, 10)
```

```
     Ttest       Sign SignedRank
         1          1          1
```

```
simnormal(40, 5, 37, 10)
```

```
     Ttest       Sign SignedRank
     0.395      0.192      0.378
```

You'd need a way to save the results, but you could do that in the function - have it loop through the settings instead of being used like this and save results as it goes.

In this setting, the data we generated followed a normal distribution, so we expected the t-test to do well, but signed rank should have been nearly comparable, according to theory. What might happen if we try some different distributions? The t-test assumes a normal distribution, so if we change it, can the nonparametric methods do better than the t-test? (The answer is yes, by the way.)

Presenting simulation study results

Presenting results from a simulation study usually means making a table (or several) or figure (or several) to display results, which you then have to discuss. Figuring out the best layout of tables is often tricky, and I encourage you to sketch it (by hand!) to figure out what makes the best sense for your setting. You usually have to convey the levels of all parameters you were varying, and then whatever summary statistic(s) you were recording from the replications.